



مشروع الخوارزميات 2

بإشراف
أ.عدنان نعمان
أ.رغدان غبرة

إعداد الطلاب :

نور الدين سردار خان الأفغاني
مروان نقلي الشهير بالداوودي

معتز فراس أبو عاصي
محمد هاني حوراني



المسألة الأولى

نقدم لكم ("**Supply Chain System**") نظام لإدارة سلسلة توريد لشركة إدارة المنتجات في المستودعات، وتخزين بيانات الشحنات، وتحديد أولويات التسليم

النظام يتضمن الخدمات التالية:

- 1) إدارة سجلات المنتجات
- 2) البحث عن المنتجات
- 3) تخزين سجلات الشحنات
- 4) إدارة أولويات التسليم

سنقوم باستخدام بنى المعطيات غير الخطية

("non Linear Data Structures") ومن أهمها والمستخدم في نظامنا

Heap و AVL و Binary Search Tree (BST)

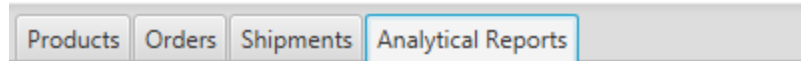
سبب استخدامنا خصيصا لهذه البنى هو زيادة أداء النظام من حيث عمليات البحث والإضافة والحذف

وانطلاقا من هذه العمليات الأساسية تطبقنا تم بناؤه.

ولكن قبل البدء أريد التنويه على أن النظام مكتوب بال Java وال Frame

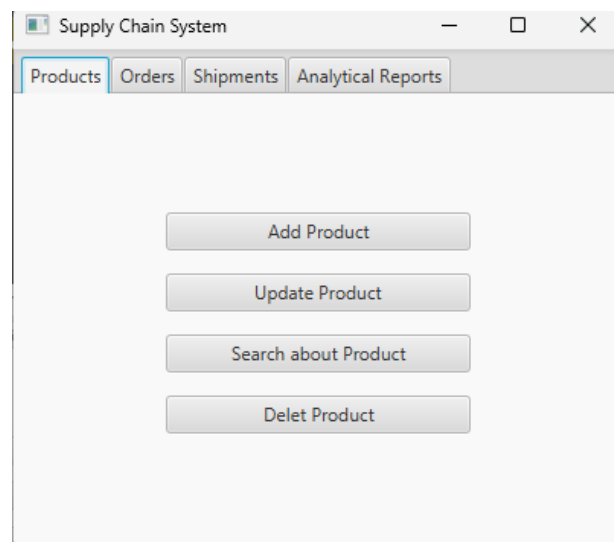
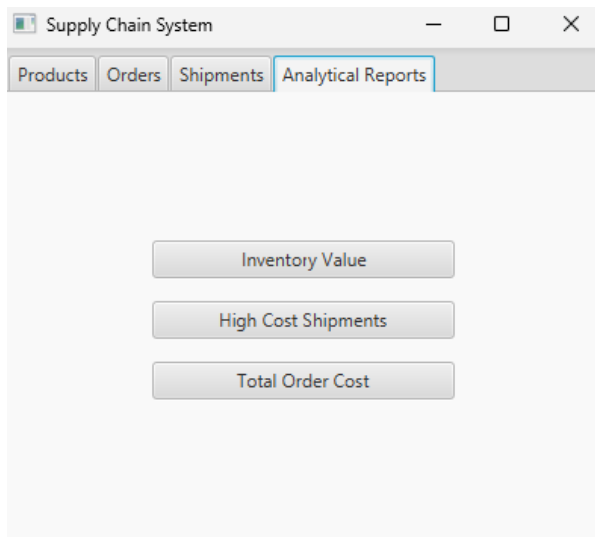
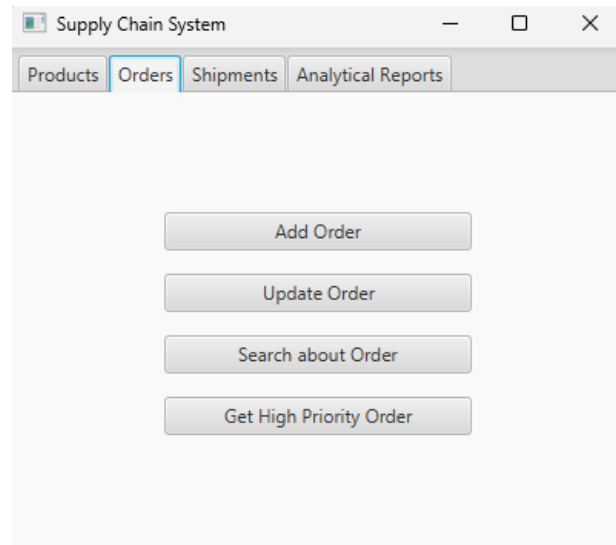
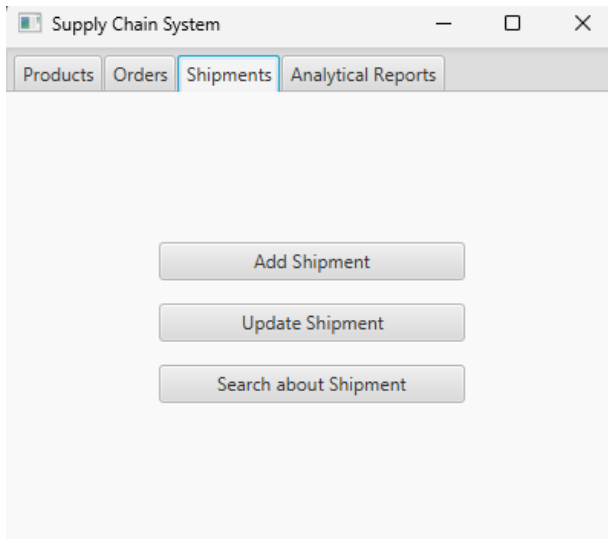
Java FX :Work

لا ننسى أننا وفرنا للمستخدم أريحية التنقل بين الخدمات عن طريق ال Tabs الموجودة في الأعلى من اليسار

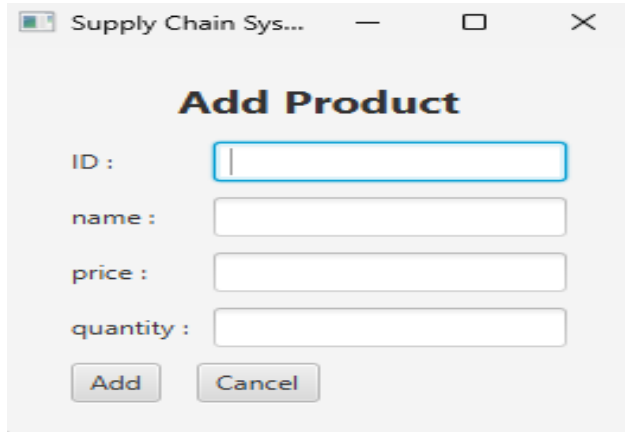


بالإضافة لواجهات توفر التنقل بين الخدمات الفرعية بين خدمات التطبيقات الأساسية

وكل ذلك لتوفير أفضل تجربة مستخدم UX



(a) نفتح الحديث عن نظامنا بما ذكرناه سابقا من خدمات ونبدأ بأول خدمة ولإنعاش الذاكرة كانت "إدارة سجلات المنتجات"



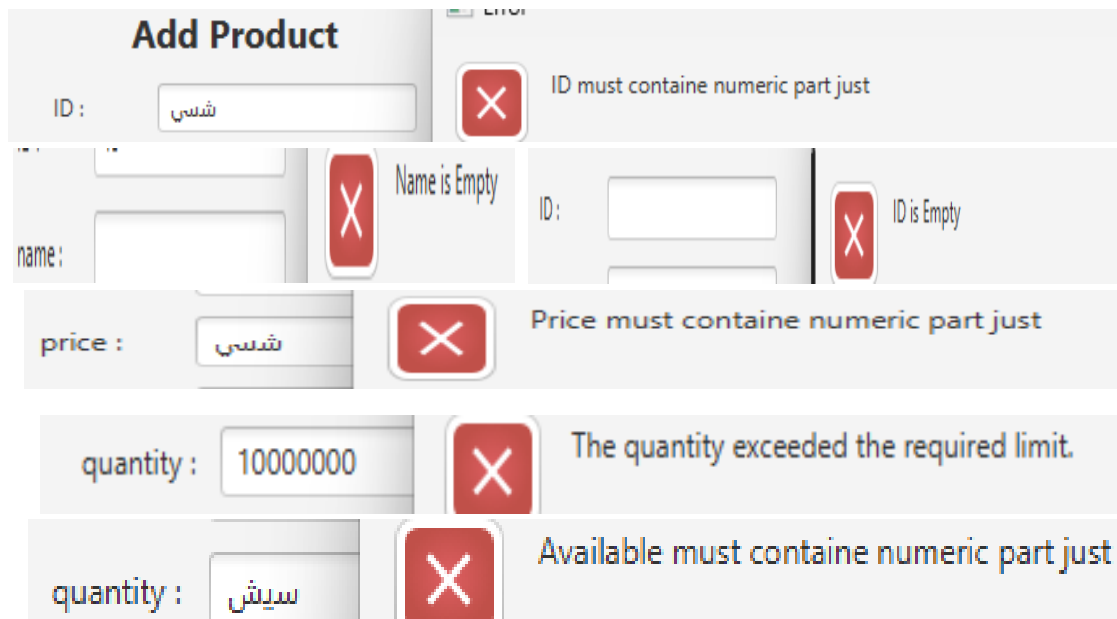
✓ فنظامنا في هذا القسم يوفر للمستخدم إضافة منتج جديد بحيث يترتب إدخال التالي

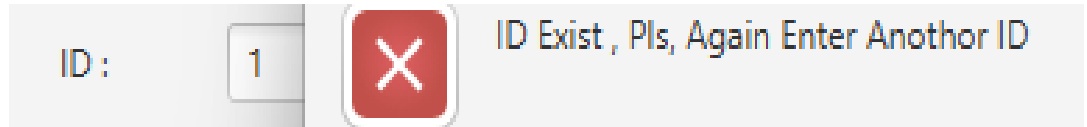
ID : من أجل توفير الوصول لأي منتج علما أنه فريد لكل منتج .

name : من أجل معرفة المنتج نظرا.

price : سعر المنتج / quantity : الكمية المتوفرة من هذا المنتج

ولا ننسى أن أهم ما في الأمر مناقشة جميع رسائل الخطأ الممكنة لإيصال المستخدم لعملية الإدخال الصحيح عن طريق توجيهه برسائل خطأ واضحة



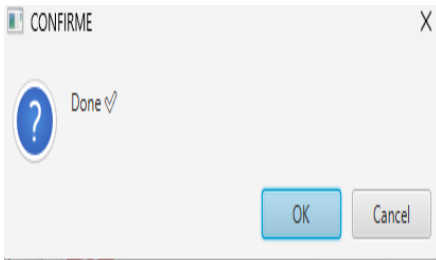


وغيرها من رسائل الخطأ.

عند الإدخال الصحيح للمنتج مثلاً

A dialog box titled "Add Product" from a "Supply Chain Sys..." application. It contains four input fields: "ID :" with value "15", "name :" with value "Bata", "price :" with value "456", and "quantity :" with value "45". At the bottom are "Add" and "Cancel" buttons.

هنا نلاحظ إظهار نافذة منبثقة للتأكد من البيانات قبل حفظها وبالموافقة عليها يتم حفظ معلومات المنتج اذا كانت خالية من الأخطاء



هنا سيتم إضافة المنتج بنجاح

بنى المعطيات المستخدمة:

1 - Binary Search Tree (BST)

لقد قمنا باستخدام بنية المعطيات أشجار البحث الثنائية لتخزين أنواع البضائع، ويتم وضع البضائع ضمن الشجرة حسب المعرف الخاص بكل نوع من البضائع.

يمكننا القيام على الشجرة بعمليات البحث والحذف والإضافة بتعقيد مقبول ولكن ليس الأمثل حيث في حالة المعرفات المتسلسلة تصبح الشجرة من فرع واحد أي كأنها سلسلة خطية.

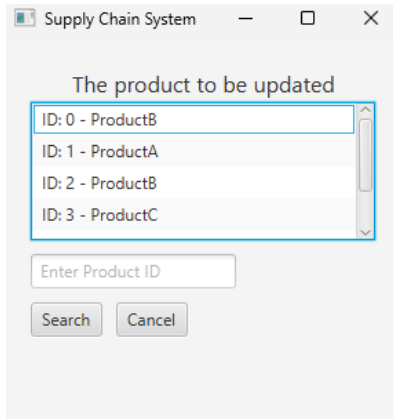
Search: avg: $O(\log n)$ worst : $O(N)$

Add: avg: $O(\log n)$ worst : $O(N)$

Delete: avg: $O(\log n)$ worst : $O(N)$

ولذلك نلجأ لاستخدام ال AVL أي الشجرة المتوازنة تلقائياً للحفاظ على أفضل تعقيد حتى في أسوء الحالات

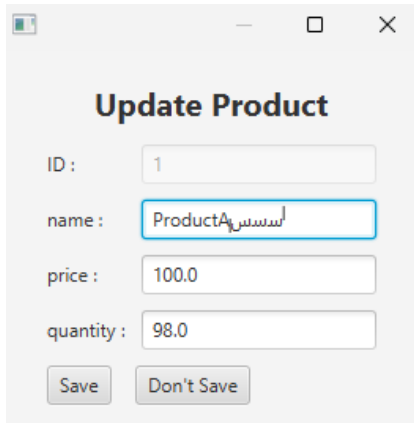
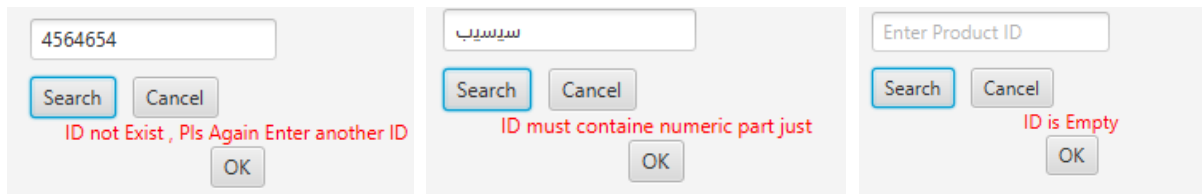
تعديل المنتجات:



بعد إدخال كل المتطلبات وظهور رسائل الخطأ المناسبة لتوجيه المستخدم لإضافة منتج مع ذكر أهم رسالة وهي ال **ID Exist** , سنستخدم خوارزميات الإضافة في ال BST

✓ ثم ننتقل لثاني خدمة فرعية وهي تحديث منتج سنبدأ بواجهة البحث عن المنتج المراد تحديثه

مع التنويه لرسائل الخطأ المرشدة بشكل واضح



وبعد إدخال ال ID المناسب ننتقل لواجهة

مزاي هذه الواجهة:

(a) توفر للمستخدم تحديث معلومات المنتج فقط

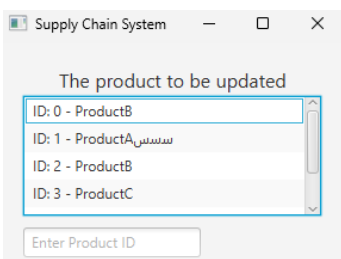
وليس ال ID

(b) وتوفر أيضا مناقشة نفس أخطاء واجهة الإضافة

المرشدة ولكن من دون أخطاء ال ID

(c) ولا ننسى التنويه على أن معلومات المنتج المراد تعديله تظهر في الحقول المناسبة للتعديل

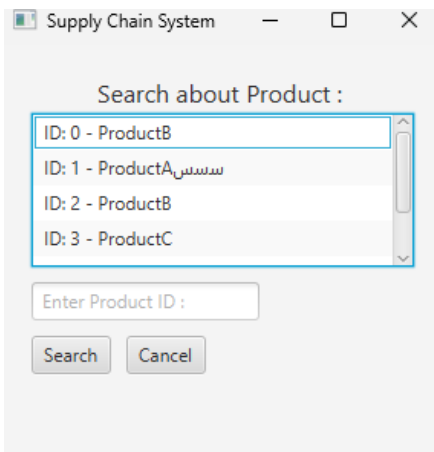
والتحديث ينعكس تلقائيا على جدول البحث الموفر لكي يتأكد المستخدم من التحديث مباشرة



وهكذا نلاحظ التعديل الحاصل.

البحث:

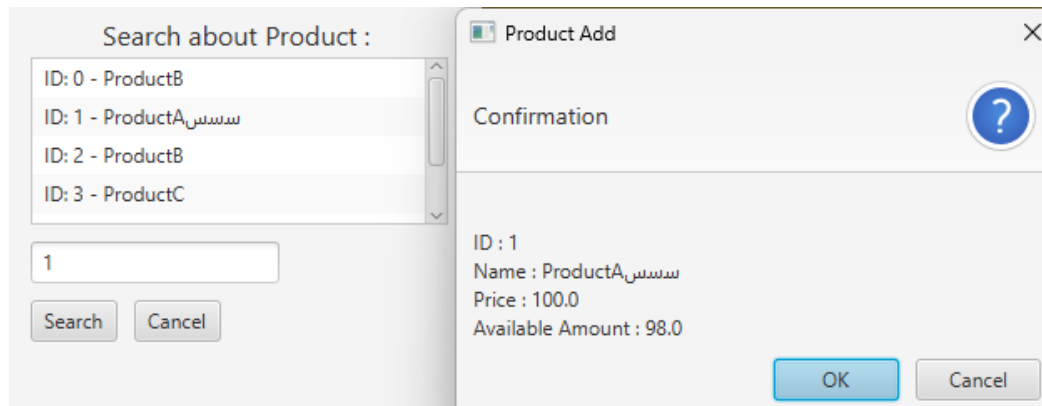
عند إدخال ال ID أقوم بمناقشة جميع الأخطاء التي تواجه المستخدم بالإدخال وعند التأكد من صحة ال ID نستخدم خوارزميات البحث في ال BST ✓ والآن سننتقل الى خدمة فرعية أخرى وهي البحث عن منتج



لقد قمنا بملاحظة أن الواجهة تشبه واجهة التحديث ولكن تختلف بالنتيجة وتتشابه في عملية مناقشة الأخطاء.

قمنا بإدخال ال رقم 1 فظهر التالي

وهكذا قمنا بالتوفير على المستخدم استعمال البحث اليدوي .



الحذف:

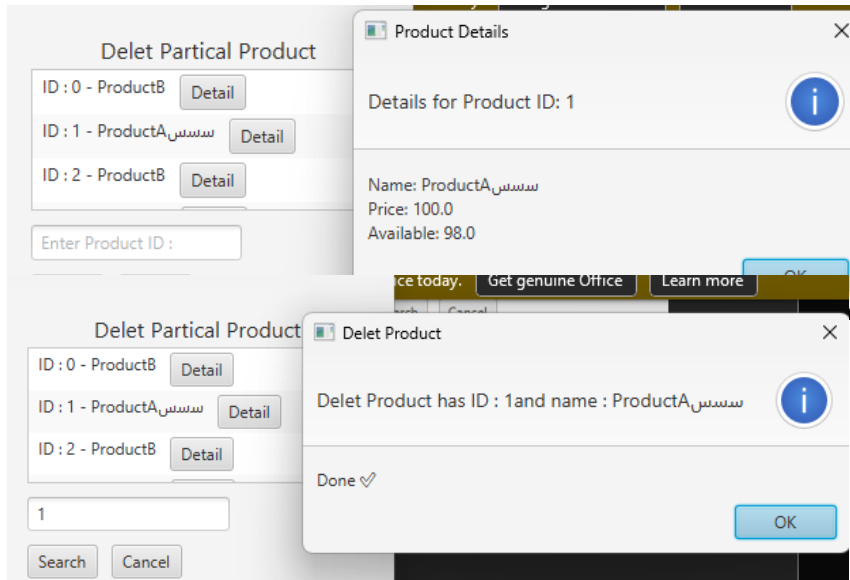
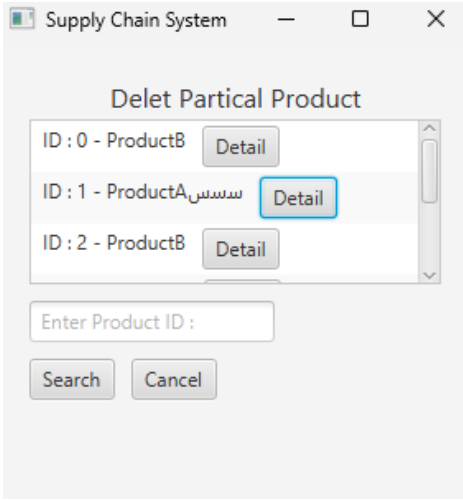
✓ ننتقل للخدمة الفرعية الأخيرة وهي حذف منتج

ستظهر هذه الواجهة التي تقوم بتوفير أقصى

الخدمات لكي يكتفي المستخدم بهذه النافذة

مستخدماً أزرار ال Detail للحصول على

معلومات المنتجات التي سيقوم ليتأكد من المنتجات المستهدفة.



فعند الضغط ع الزر

تظهر النافذة المنبثقة

التي توفر جميع تفاصيل
المنتج.

مع العلم قمنا بتوفير

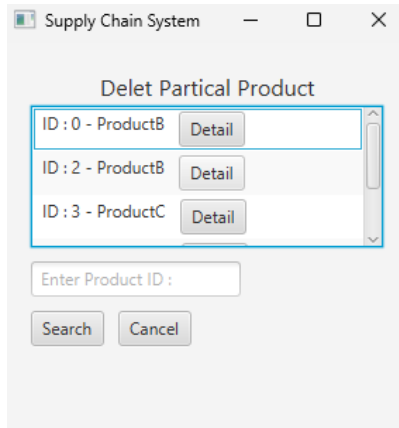
جميع رسائل الخطأ

المناسبة لتوجيه المستخدم

كما الواجهات السابقة

المشابهة لهذه الواجهة.

بالإضافة للتأكيد على المستخدم بحذف المنتج عن طريق إعادة عرض المنتج
المحذوف.



وهكذا تم حذف المنتج والذي يؤكد النتيجة هي ترتيب ال IDs بحيث أصبح 0 , 2 , 3 ,

قبل الانتقال للخدمة الأساسية الثانية سننوه لوجود مشكلة كبيرة في بنية المعطيات BST وهو أن عند إدخال عدة معرفات متتالية أي (0,1,2,3,4,5,.....)

بحسب قانون إضافة عقدة لـ BST سوف يتم تموضعهم بشكل خطي مما سيؤدي لزيادة التعقيد وتقليل الأداء لأن تعقيد سيتحول $O(\log n)$ - $O(n)$ وهذا ما نقوم بالابتعاد عنه في نظامنا فقمنا باستبدال class BST بـ class AVL .
ما هي ال AVL : شجرة متوازنة ذاتيًا تحسن من خوارزميات البحث في جميع الأحوال.

فقد قمنا بربط نفس الواجهة مع ال AVL لتحسين التعقيد بما أن برنامجنا يتعامل مع نوع البيانات بدinamikiكية وبتأقلم كامل معه عند التغيير من أحد أنواع الأشجار لأخر.

الطلبات:

(b) سننتقل الى واجهة الطلبات والتي هي "توفر خدمة إدارة أولويات تسليم الطلبات"

✓ فنظامنا في هذا القسم يوفر للمستخدم إضافة طلب جديد بحيث يترتب إدخال التالي

مع العلم الطلبات يتم ترتيبها حسب الأولوية

Order Name : من أجل معرفة الطلب نظرا.

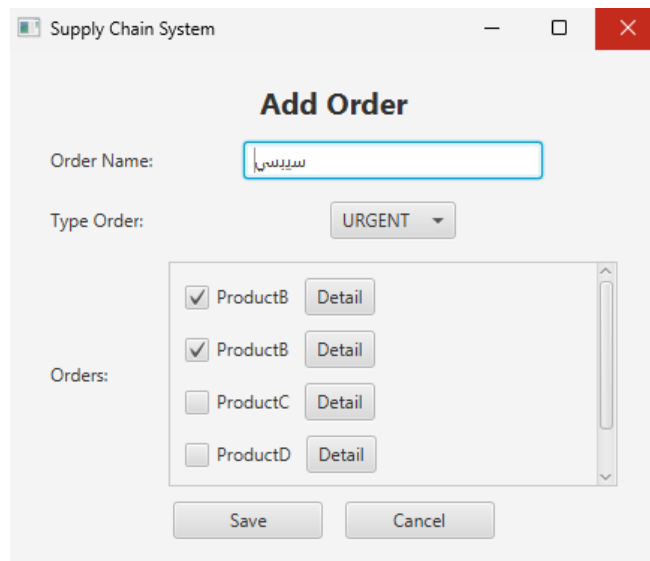
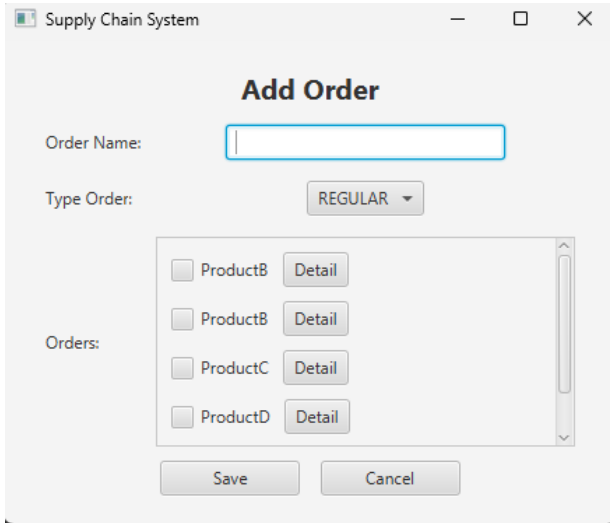
Type Order : من اجل تحديد أولوية الطلب

Products : من أجل عرض المنتجات

المتوفرة إضافتها للطلب.

مع مناقشة حدود الميزانية عن طريق تحديد أعلى مبلغ لكل الطلبات.

ولا ننسى أهم ما في الأمر مناقشة جميع رسائل الخطأ الممكنة لإيصال المستخدم لعملية الإدخال الصحيح عن طريق توجيهه برسائل خطأ واضحة



بعد الإدخال الكامل

3 - Max-Heap

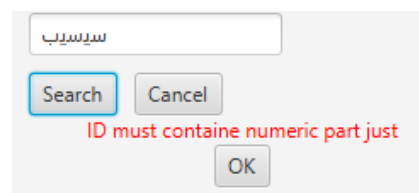
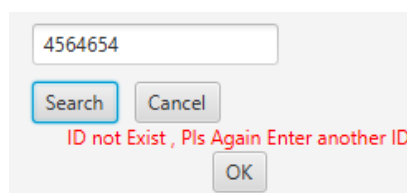
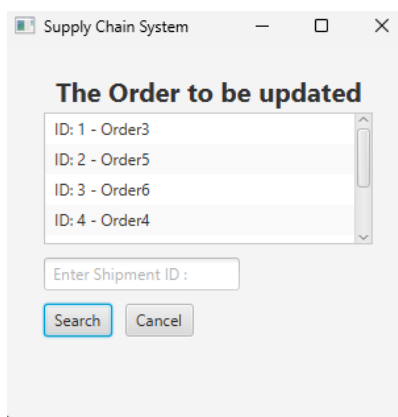
شرح ما جرى:

بعد إدخال كل المتطلبات وظهور رسائل الخطأ المناسبة لتوجيه المستخدم لإضافة طلب مع ذكر أهم رسالة وهي ال **ID Exist** ، سنستخدم خوارزميات الإضافة.

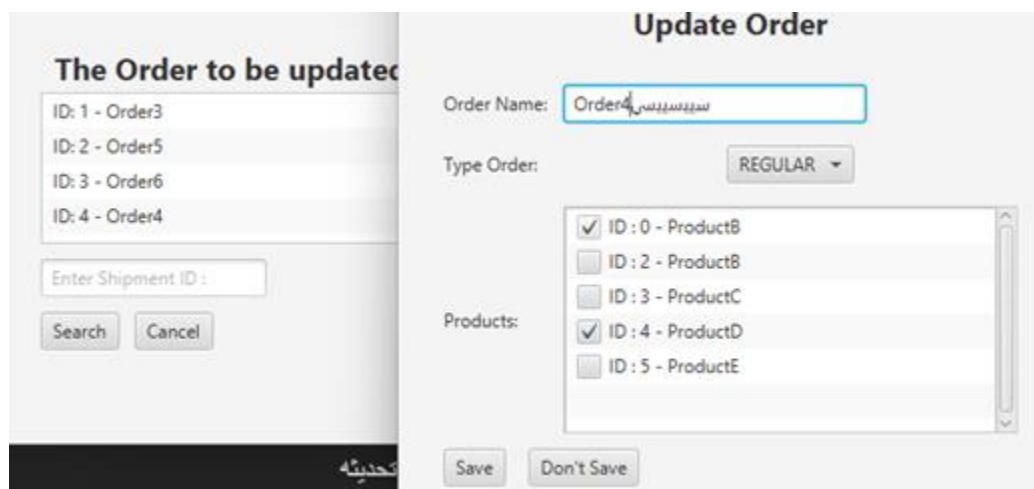
✓ ثم ننتقل لثاني خدمة فرعية وهي تحديث طلب

سنبدأ بواجهة البحث عن الطلب المراد تحديثه

مع التنويه لرسائل الخطأ المرشدة بشكل واضح



وبعد إدخال ال ID المناسب ننتقل لواجهة

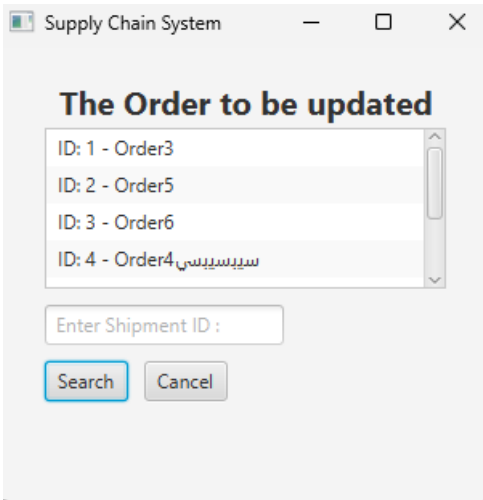


مزايا هذه الواجهة:

- (a) توفر للمستخدم تحديث معلومات الطلب
- (b) مناقشة نفس أخطاء واجهة الإضافة المرشدة
- (c) التنويه على أن معلومات الطلب المراد تعديله تظهر في الحقول المناسبة للتعديل
- (d) إضافة لإعادة ترتيبه في جدول العرض الخاص بالطلبات في حال فقط تم تغيير الأولوية.

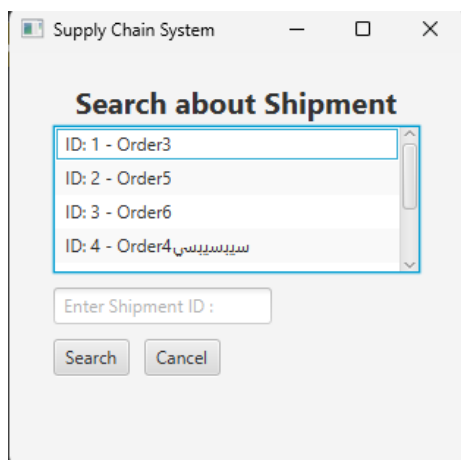
والتحديث ينعكس تلقائيا على جدول البحث الموفر لكي يتأكد المستخدم من التحديث مباشرة

وهكذا نلاحظ التعديل الحاصل.



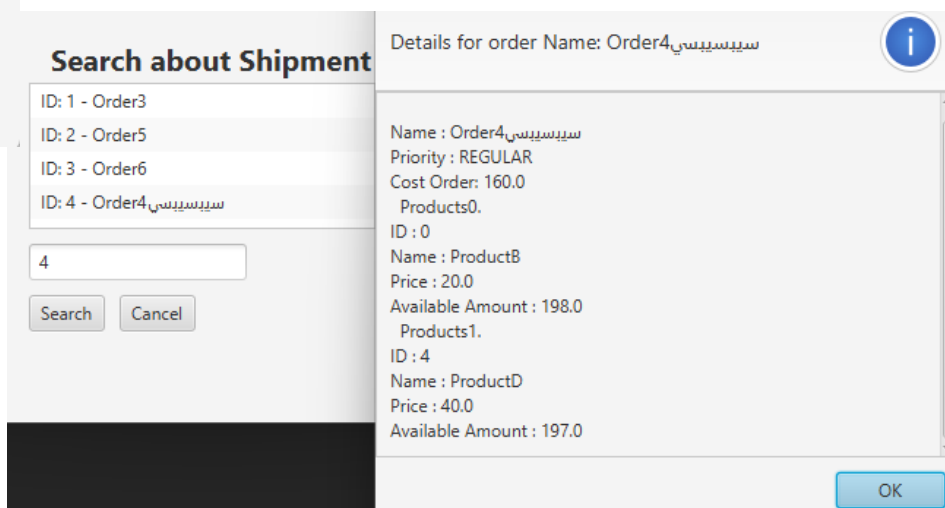
عند إدخال ال ID أقوم بمناقشة جميع الأخطاء التي تواجه المستخدم بالإدخال وعند التأكد من صحة ال ID نستخدم خوارزميات البحث في ال Max Heap ونقوم بالوصول لقيم حقول الإدخال وإعادة إسنادها للمنتج المراد تحديثه.

✓ والآن سننتقل الى خدمة فرعية أخرى وهي البحث عن طلب



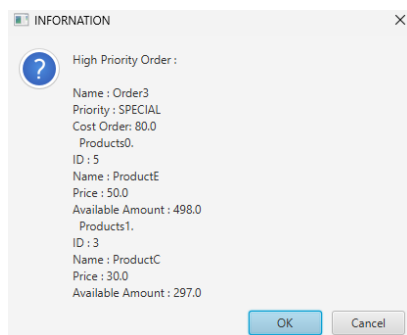
لقد قمنا بملاحظة أن الواجهة تشبه واجهة التحديث ولكن تختلف بالنتيجة وتتشابه في عملية مناقشة الأخطاء.

قمنا بإدخال ال رقم 4 فظهر التالي



وهكذا قمنا بالتوفير على المستخدم استعمال البحث اليدوي.

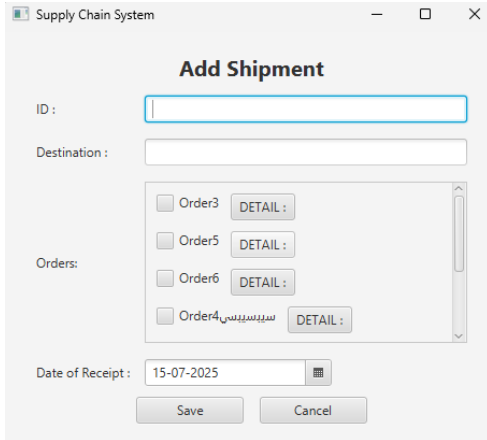
عند إدخال ال ID أقوم بمناقشة جميع الأخطاء من تواجه المستخدم بالإدخال وعند التأكد من صحة ال ID نستخدم خوارزميات البحث في ال PQ فنصل للطلب ونقوم بعرضه علما أن الوصول $O(1)$ لأنني أصل ل index في Array
✓ ننتقل للخدمة الفرعية الأخيرة وهي عرض الطلب ذو الأولوية العالية



عند الضغط ع الزر الخاص بهذه المهمة يعرض التالي

عند إدخال ال ID أقوم بمناقشة جميع الأخطاء التي تواجه المستخدم بالإدخال وعند التأكد من صحة ال ID نستخدم خوارزميات البحث في ال Max Heap من أجل حذف العقد المطلوبة.

c) سننتقل الى واجهة الشحنات والتي هي "توفر تخزين سجلات الشحنات " ✓ فنظامنا في هذا القسم يوفر للمستخدم إضافة شحنة جديد بحيث يترتب إدخال التالي



ID : من أجل توفير الوصول لأي طلب علما أنه

فريد لكل منتج.

Destination: من أجل معرفة وجهة الشحنة.

Orders : من أجل عرض الطلبات المتوفرة

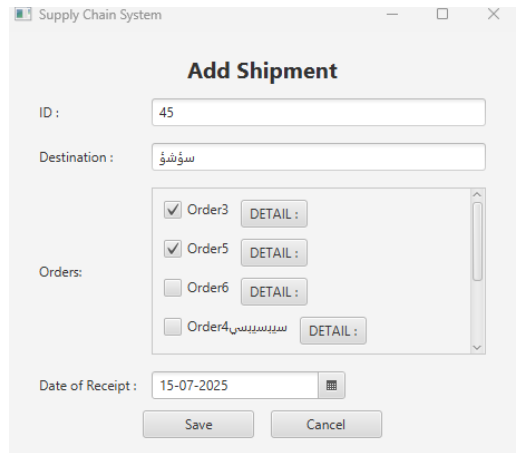
إضافتها للشحنة.

مع مناقشة حدود الميزانية عن طريق تحديد أعلى مبلغ لكل الشحنات.

ولا ننسى أهم ما في الأمر مناقشة جميع رسائل الخطأ الممكنة لإيصال المستخدم لعملية الإدخال الصحيح عن طريق توجيهه برسائل خطأ واضحة

Date of Reciept : يسمح للمستخدم بتحديد تاريخ تسليم الشحنة مع مناقشة عدم إدخال التواريخ القديمة

بعد الإدخال الكامل:

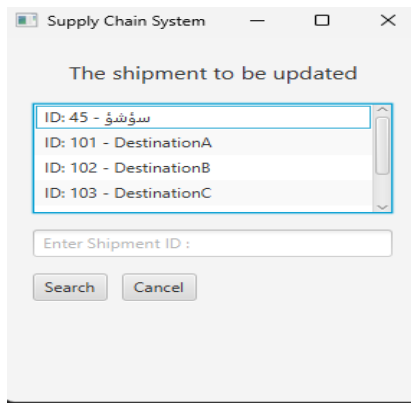


بنى المعطيات المستخدمة:

AVL

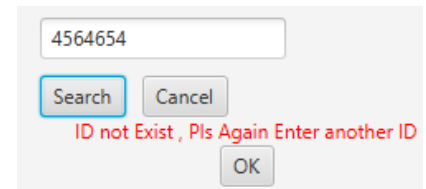
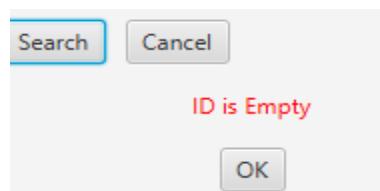
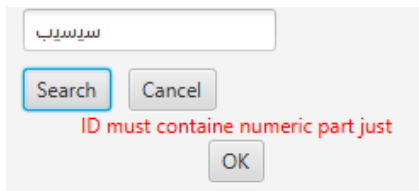
شرح ما جرى:

بعد إدخال كل المتطلبات وظهور رسائل الخطأ المناسبة لتوجيه المستخدم لإضافة طلب مع ذكر أهم رسالة وهي ال **ID Exist** ، سنستخدم خوارزميات الإضافة في ال AVL

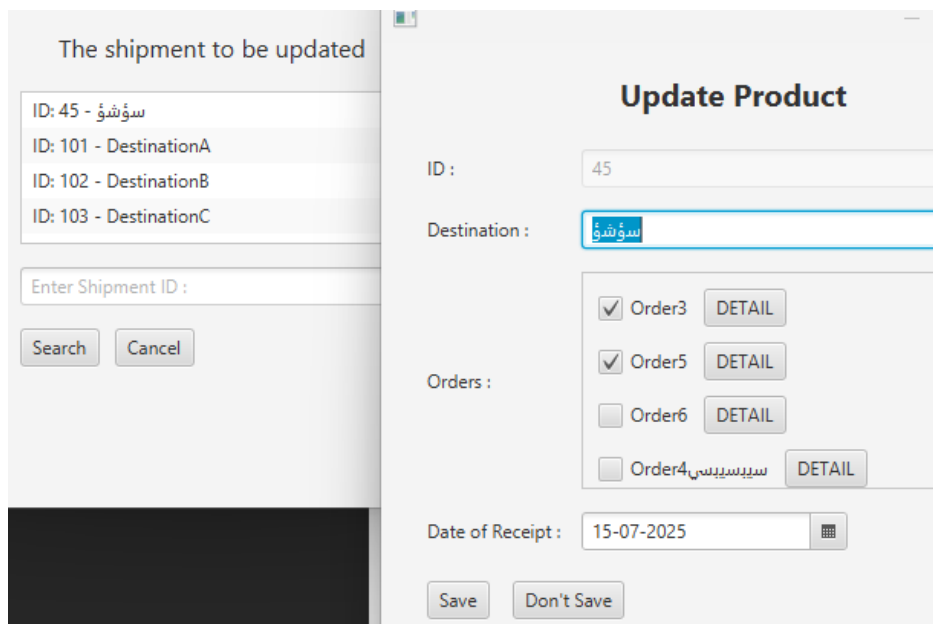


✓ ثم ننتقل لثاني خدمة فرعية وهي تحديث الشحنة
سنبدأ بواجهة البحث عن الشحنة المراد تحديثها

مع التنويه لرسائل الخطأ المرشدة بشكل واضح



وبعد إدخال ال ID المناسب ننتقل لواجهة



مزايا هذه الواجهة:

1. توفر للمستخدم تحديث معلومات الشحنة
 2. مناقشة نفس أخطاء واجهة الإضافة المرشدة
 3. التنويه على أن معلومات الشحنة المراد تعديله تظهر في الحقول المناسبة للتعديل
 4. يمنع إدخال تاريخ قديم
- والتحديث ينعكس تلقائيا على جدول البحث الموفر لكي يتأكد المستخدم من التحديث مباشرة

وهكذا نلاحظ التعديل الحاصل.

عند إدخال ال ID أقوم بمناقشة جميع الأخطاء التي تواجه المستخدم بالإدخال وعند التأكد من صحة ال ID نستخدم خوارزميات البحث في ال AVL ونقوم بالوصول لقيم حقول الإدخال وإعادة إسنادها للشحنة المراد تحديثه.

✓ والآن سننتقل الى خدمة فرعية أخرى وهي البحث عن طلب

لقد قمنا بملاحظة أن الواجهة تشبه واجهة التحديث، ولكن تختلف بالنتيجة وتتشابه في عملية مناقشة الأخطاء.

Supply Chain System

The shipment to be updated

ID: 45 - سؤشؤلاتلات
ID: 101 - DestinationA
ID: 102 - DestinationB
ID: 103 - DestinationC

Enter Shipment ID :

Search Cancel

Supply Chain System

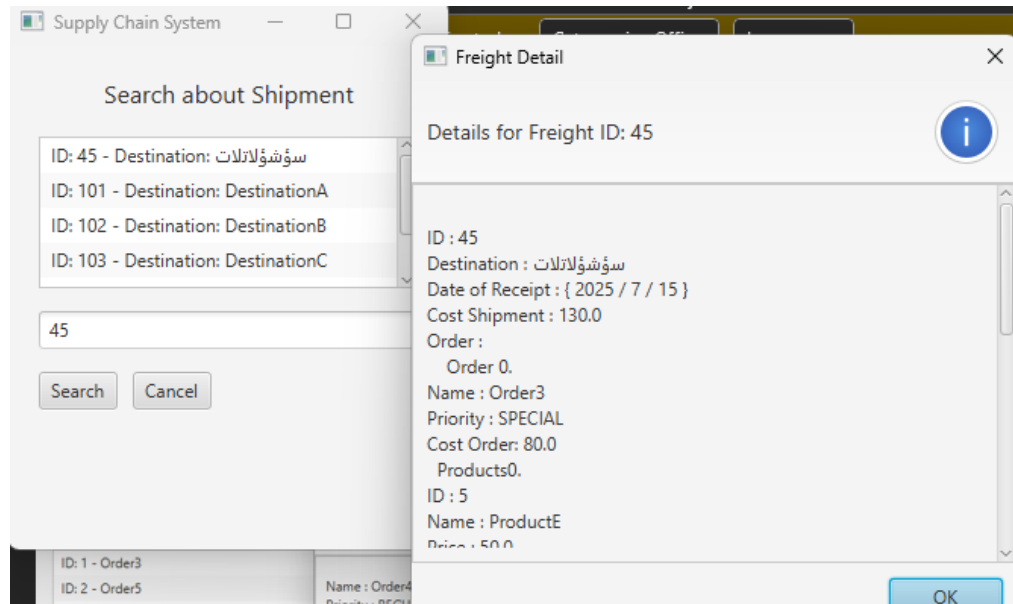
Search about Shipment

ID: 45 - Destination: سؤشؤلاتلات
ID: 101 - Destination: DestinationA
ID: 102 - Destination: DestinationB
ID: 103 - Destination: DestinationC

Enter Shipment ID :

Search Cancel

قمنا بإدخال ال رقم 4 فظهر التالي



وهكذا قمنا بالتوفير على المستخدم استعمال البحث اليدوي.

شرح ما جرى:

عند إدخال ال ID أقوم بمناقشة جميع الأخطاء التي تواجه المستخدم بالإدخال وعند التأكد من صحة ال ID نستخدم خوارزميات البحث في ال AVL فنصل للشحنة ونقوم بعرضه.

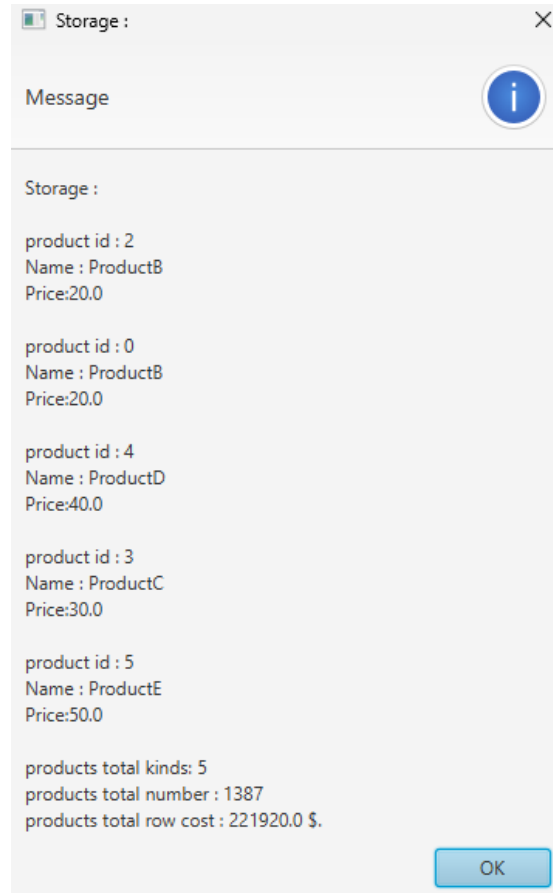
(d) سننتقل الى واجهة التقارير التحليلية

تنقسم ل 3 أزرار

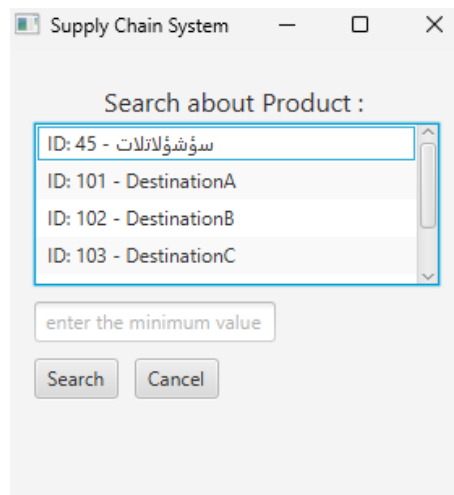
الأول : يعرض قيمة المخزون

أي يعرض جميع المنتجات من المستودع من حيث الاسم والسعر ويعطي

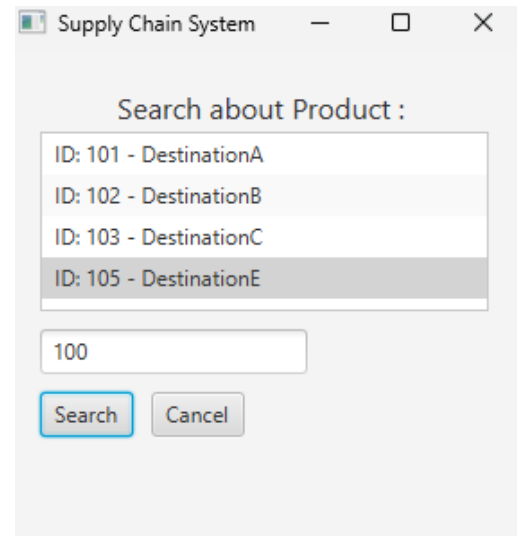
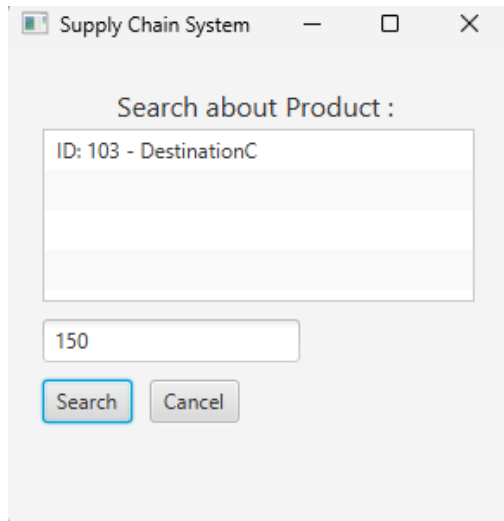
التكلفة النهائية للمخزون ويحفظ المعلومات الى ملف statistics



الثاني: يقوم بعرض صفحة بحث يعرض فيها كل الشحنات ويوجد حقل إدخال لحد من أجل تحديد الشحنات المرادة (عالية التكلفة) التي تتجاوز قيمتها القيمة الحدية

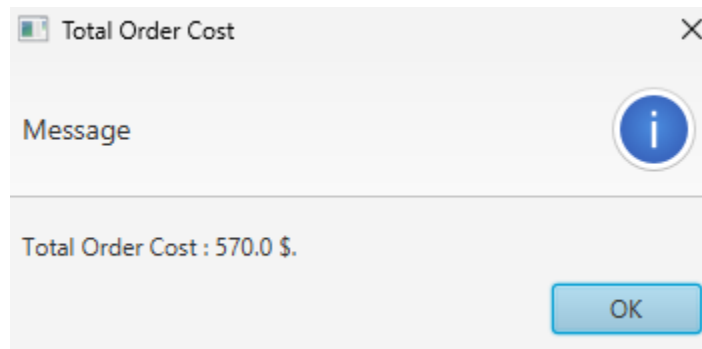


واجهة البحث بعد ادخال القيمة الحدية تظهر الشحنات عالية السعر أي الشحنات ذات السعر الأكبر من السعر المدخل



الثالث:

يعطيني إجمالي تكلفة الطلبات عن طريق التجول على الطلبات وحساب أسعارهم



ختاماً:

ومنه نختتم الحديث عن نظامنا وما يقدمه من تحسينات لتجربة المستخدم وبهذا نظامنا يقدم لكم

- (a) التحقق من القيود
(b) انشاء تقارير تحليلية
(c) تحديث بيانات ديناميكية
(d) التعامل مع الحالات الخاصة (منع التواريخ الماضية وحدود الميزانية)
- وبهذا نكون قد غطينا جميع الطلبات الأساسية والإضافية لهذا البرنامج.

المسألة الثانية:

النص:

لدينا جزيرة مُسطيلة بحجم $m \times n$ يحدها المحيط الهادي والمحيط الأطلسي. يلامس المحيط الهادي حواف الجزيرة اليسرى والعلوية، بينما يلامس المحيط الأطلسي حواف الجزيرة اليمنى والسفلى. تم تقسيم الجزيرة إلى شبكة من الخلايا المربعة. تهطل على الجزيرة الكثير من الأمطار، ويمكن لمياه الأمطار أن تتدفق إلى الخلايا المجاورة شمالاً وجنوباً وشرقاً وغرباً في حال كان ارتفاع الخلية المجاورة أقل من ارتفاع الخلية الحالية. ويمكن للماء أن يتدفق من أي خلية مجاورة للمحيط إلى المحيط. اطبع مصفوفة "result" حيث $ci, result_i = ri$ تدل على أن مياه الأمطار يمكن أن تتدفق من الخلية " (ci, ri) " إلى كل من المحيط الهادي والمحيط الأطلسي.

بنى المعطيات المستخدمة:

Directed unweighted graph

المسألة عبارة عن مسألة بيان موجه

حيث كل خلية هي عبارة عن عقدة vertex ضمن الغراف

البيان موجه هنا لأنه يمكن الانتقال من الخلية ذات الارتفاع الأعلى للأخفض ولكن ليس

بالعكس

البيان غير موزون لأننا نهتم بالانتقال من خلية لأخرى مجاورة لها ذات ارتفاع أقل ولا نهتم

بمسافة الطريق أو الزمن اللازم للماء حتى ينتقل من خلية لأخرى.

لا يوجد تكلفة للانتقال من خلية لأخرى فقط نهتم بكل خلية بالخلايا المجاورة والتي يستطيع

الماء الانتقال إليها

تم تمثيل هذا البيان بأسلوبين:

الأسلوب الأول والأبسط

كل خلية هي عقدة اي لدينا $v = n * m$

ونمثل البيان على مصفوفة ثنائية من البعد v

`boolean graph[v][v]`

بحيث تمثل العبارة

`graph[i][j]` امكانية الانتقال من العقدة i الى j

اي وجود او غياب الوصلة بين هاتين العقدتين

يوجد وصلة بين العقدتين في حالة كانتا متجاورتين وارتفاع واحدة أكبر من ارتفاع الأخرى ويكون جهة الوصلة من العقدة الأكبر ارتفاع باتجاه الأصغر ارتفاعا و(في حال النهر تكون الوصلة بالاتجاهين)

فيما يلي المثال التالي والمصفوفة الثنائية (البيان) الممثل له:

2	3	5
3	4	4
5	3	1

ويكون البيان الممثل له بالشكل:

from/to	0	1	2	3	4	5	6	7	8
0	0	0	0	0	0	0	0	0	0
1	1	0	0	0	0	0	0	0	0
2	0	1	0	0	0	1	0	0	0
3	1	0	0	0	0	0	0	0	0
4	0	1	0	1	0	0	0	1	0
5	0	0	0	0	0	0	0	0	1
6	0	0	0	1	0	0	0	1	0
7	0	0	0	0	0	0	0	0	1
8	0	0	0	0	0	0	0	0	0

وبقراءة الجدول وبملاحظة أن سطر كل عقدة يمثل العقد التي تستطيع الانتقال لها نجد أن العقدة

$x \rightarrow 0$

$0 \rightarrow 1$

$5 \rightarrow 2, 1 \rightarrow 2$

$7 \rightarrow 4, 3 \rightarrow 4, 1 \rightarrow 4$

وهكذا نستطيع الوصول للوصلات كلها بسهولة بالمرور على سطر كل خلية ولكن نلاحظ تخزين العديد من الخلايا التي لم استخدمها ولذلك نلجأ للطريقة الثانية لتخفيف التعقيد المكاني للخوارزمية

الطريقة الثانية للتمثيل:

`ArrayList<LinkedList<Integer>>`

وبهذه الطريقة نخزن فقط الوصلات التي نستطيع الوصول لها تمثل ال `ArrayList` الخارجية هي العقد التي آتي منها `from` وتكون قيم سلسلة التجاور عندها هي ارقام العقد التي أستطيع الوصول لها من العقدة `i` في ال `arraylist`

وبهذا ننتهي من قسم بنى المعطيات.

الخوارزمية المستخدمة وطريقة الحل:

- 1 - نقوم بإنشاء البيان المناسب من الدخل كما رأينا في القسم السابق
- 2- المرور على جميع العقد وحساب فيما إذا كانت توصل المياه الى المحيطين ام لا
- 3- رسم البيان المناسب باستخدام swing

طريقة الحل:(من دون حالة الانهار)

اولا أحد التوابع المهمة هنا هو توابع لتحديد إذا كانت العقدة توصل لاحد المحيطين
متى العقدة توصل للمحيط الهادي؟

إذا كانت العقدة اسطرها في السطر 0 او اعمدتها في العمود 0
ويمكن حساب سطر كل عقدة او عمودها من البيان الناتج بالطريقة التالية

$$\text{row} = i/m$$

$$\text{column} = i \% m$$

حيث ال i هو رقم العقدة في مصفوفة البيان

وال m عدد اعمدة مصفوفة الدخل الاساسية

وتكون الخلية توصل للمحيط الاطلسي إذا كانت في اخر سطر او اخر عمود

ثانيا تابع الحل الاساسي

يقوم التابع بالمرور من العقدة الى العقد التي توصل لها بشكل عودي بطريقة dfs وحساب
قيمة integer لكل خلية وحسابها

وهذه القيمة تمثل امكانية الخلية بالوصول الى أحد المحيطين او كلاهما ويتم تخزينها بمصفوفة
بحيث نحسب هذه القيمة لكل خلية مرة واحدة فقط لا غير للحفاظ على اقل تعقيد ممكن $O(V)$

قيمة ال integer هي

1 محيط أطلسي

2 محيط هادي

3 كلا المحيطين

وتنتهي مهمة هذا التابع حالما يحسب قيمة 3 او انتهاء جميع الطرق التي توصل لها الخلية

اي يقوم التابع عند استدعائه للخلية الاولى باستدعاء عودي لنفسه عند الخلايا المجاورة لها ذات الارتفاع الاصغر منها اي كل عنصر من سطر الخلية الاولى في حال لم نحسب القيمة للخلايا من قبل

ويكون شرط التوقف له هو في حالة كانت القيمة محسوبة في مصفوفة المحيطات ويعيد هذه القيمة مباشرة او في حال لم تكن محسوبة يتوقف عندما يصل لخلية تطل على كلا المحيطين او تنتهي الطرق الممكنة للذهاب لها من الخلية ويرجع قيمة مناسبة 1 2 او 3 حسب المحيطات التي استطاع الوصول لها

ثالثا تابع للمرور على العقد واستدعاء تابع الحل لها وبالتالي ملئ مصفوفة المحيطات

رابعا رسم الغراف
نملك في مصفوفة المحيطات قيم 3 وتمثل العقد التي توصل لكلا المحيطين اي الحل المطلوب فقط نحتاج لرسمه
باستخدام java swing
قمنا بإنشاء labels بعدد الخلايا ورتبناها ضمن جدول وعرضناها

الطلبات الإضافية:

- 1- انشاء مصفوفة للصحراء بحيث في تابع الحل الاساسي عند الوصول لصحراء تتسرب جميع المياه عبر الرمل (تتبخر) ولا تكمل مسيرها الى المحيطات
الحل لهذا الطلب بسيط لأنه يتوافق مع ما سبق
- 2- بالنسبة لطلب الانهار ففي حالة جميع العقد تملك نفس الارتفاع تصبح ال dfs غير جديرة بالحل ويجب أن ننقل لتفكير اخر معاكس

الحل باستخدام BFS:

هنا نحن بدلا من البدء من الخلايا نبدأ من كل من المحيطين ونتجول العقد المطلة على أحد المحيطين أولا ثم على العقد المطلة على هذه العقد باستخدام Queue ومصفوفات لتحديد الزيارة ورود زيارة العقدة في كلا المصفوفتين يوصل انها حل.

ننتبه هنا أننا نحتاج graph معكوس نسجل فيه الوصول من العقد الاصغر الي الأكبر حتى نستطيع المرور من المحيطات الي الخلايا

في ملفات المشروع نجد ثلاث مجلدات أساسية:

Graph_Array

ويحوي الطريقة الأولى للحل عن طريق مصفوفة ثنائية بتعقيد ذاكرة كبير

Graph

ويحوي الطريقة الثانية للحل باستخدام `ArrayList<LinkedList> graph` ذات تعقيد أفضل بشكل عام

Gui

ويحوي ملفات الواجهة

Test

ويحوي ملفات الاختبارات المناسبة وعن طريق نسخ محتواها ولصقه في واجهة ال terminal نستطيع تجريب البرنامج بسهولة دون التطرق للإدخال كل مرة

سنشرح فقط الطريقة الثانية كونها اعقد من الطريقة الاولى:

```
int numberOfNodes;  
int n;  
int m;  
ArrayList<LinkedList<Integer>> arrRows;  
ArrayList<LinkedList<Integer>> invertedArrRows; //go from  
int[] booleanValues;  
boolean[][] desert;  
boolean[][] rivers;  
public boolean useRivers = false;
```

هذه هي ال attributes الخاصة بهذا الصف وهي تمثل عدد الخلايا ضمن البيان عدد الاسطر والاعمدة في مصفوفة الادخال
مصفوفة الغراف الاساسية لطريقة dfs وهي `arrRows`
مصفوفة الغراف للطلب الإضافي للأنهار لطريقة bfs وهي `invertedArrRows`
`BooleanValues` مصفوفة تمثل الخلايا التي توصل الي أحد او كلا المحيطين اي مصفوفة الزيارة
مصفوفة الصحراء وتمثل الخلايا التي تكون منطقة صحراوية فيتسرب المياه كاملة فيها الي التربة اي لا تذهب للخلايا المجاورة

مصفوفة الانهار وتمثل وصلات الانهار بين رقم كل عقدة بالغراف والعقدة الأخرى الي يوجد نهر اليها
متحول بولياني ليحدد الحل بالطلب الاضافي للأنهار او لا

```
Graph(int n, int m)
Graph(int n, int m, int[][] dataArray)
Graph(int n, int m, int[][] dataArray, boolean rivers[][])
```

عدة بواني للصف لتحديد نوع الحل مع الطلب الاضافي او لا

```
public void arrayEntry(int[][] dataArray)
```

تابع يقوم بأخذ مصفوفة الدخل وينشأ الغراف المناسب منها

```
public void add(LinkedList<Integer> row, int i, int nodeNumber, int value1, int value2)
```

تابع يستخدم في التابع السابق لإضافة وصلة موجهة بين الخلايا المتجاورة التي توصل المياه

```
public void addRiver(int ifrom, int jfrom, int ito, int jto)
```

تابع لإضافة نهر موجه

```
public void solve()
```

تابع الذي يستخدم توابع bfs وال dfs حسب نوع المسألة مع انهار او بدون

```
public int calculateBooleanValue(int i, int from)
```

تابع ال dfs الاساسي

```
public int addBooleanValues(int a, int b)
```

تابع يجمع قيم الbooleanValues بحيث يهتم في الحالات المناسبة حيث لا يجمع 1 مع 1
مثلا اذ ان 1 تدل على الوصول للمحيط الأطلسي

```
public void solveWithBFS()
```

تابع الحل باستخدام ال bfs في حالة استخدام الانهار

```
private void bfs(Queue<Integer> queue, boolean[] visited)
```

```
public static boolean isOnPacificOcean(int i)
```

تابع بولياني لمعرفة إذا كانت العقدة تطل على المحيط الهادي مباشرة كما تكلمنا سابقا

```
public static boolean isOnAtlanticOcean(int i)
```

معرفة الخلايا المطلة على الاطلسي مباشرة

```
public static int getRow(int i)
```

حساب السطر للخلايا ضمن الغراف

```
public static int getColumn(int i)
```

حساب العمود للخلايا ضمن الغراف

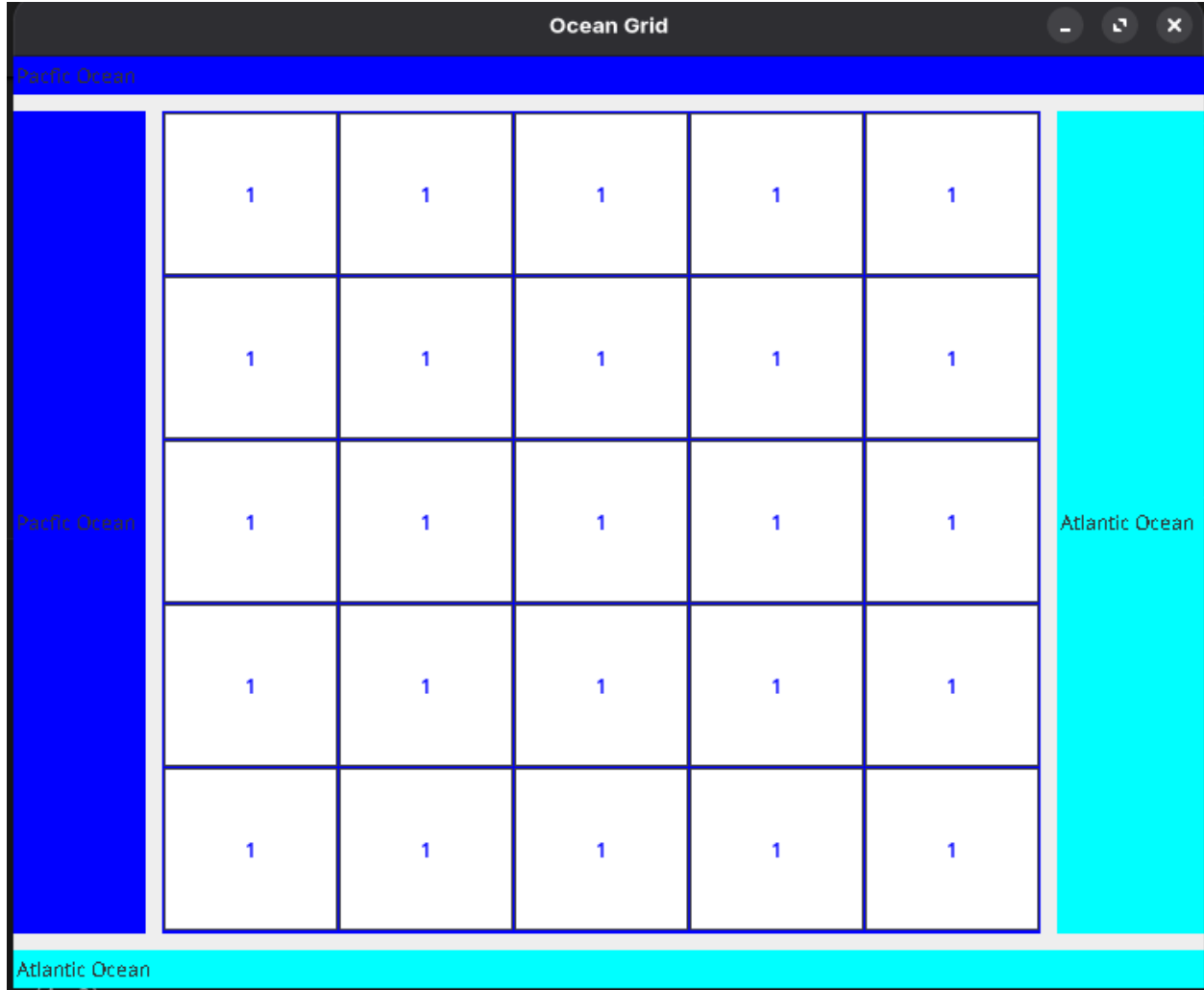
```
public static boolean adjastant(int i, int j, int i2, int j2)
```

تابع يتحقق من ان الخلايا المدخلة لوضع نهر بينها متجاورة قبل اضافة النهر

بعض الاختبارات:

1- حالة الانهار وجميع الخلايا لها نفس القيمة

هنا نلاحظ وجود الخط للخلايا باللون الازرق اي انها توصل لكلا المحيطين
ونلاحظ لون الحواف بين الخلايا ازرق يدل على وجود نهر



2- الاختبار الاساسي بدون انهار او صحراء

Ocean Grid

Pacific Ocean

Pacific Ocean

Atlantic Ocean

1	2	2	3	5
3	2	3	4	4
2	4	5	3	1
6	7	1	4	5
5	1	1	2	4

Atlantic Ocean

3- الاختبار الاساسي بدون صحراء مع وجود انهار بين جميع العقد المتجاورة

Pacific Ocean					
Pacific Ocean	1	2	2	3	5
	3	2	3	4	4
	2	4	5	3	1
	6	7	1	4	5
	5	1	1	2	4
Atlantic Ocean					

4- الاختبار الأساسي مع انهار ،وصحراء في السطر الاول كاملا

Ocean Grid

Pacific Ocean

1

2

2

3

5

3

2

3

4

4

2

4

5

3

1

6

7

1

4

5

5

1

1

2

4

Atlantic Ocean

Pacific Ocean

Atlantic Ocean

وبهذا نكون قد انجزنا جميع الطلبات المطلوبة مع الواجهات بأكثر من طريقة.



عمل الطلاب:

نور الدين سردار خان الأفغاني

معتز فراس أبو عاصي

مروان نقلي الشهير بالداوودي

محمد هاني حوراني

